SpendWiser

MERN Stack Financial Management System

Project Documentation — Technical Overview

Technology Stack

React 19 | TypeScript | Vite | Tailwind CSS | Node.js | Express
MongoDB | Mongoose | JWT Authentication | bcryptjs | Google Gemini AI
Recharts | ExcelJS | node-cron | Twilio (SMS) | Vercel Deployment

---

Prepared for: Technical Interview Reference
Developed by: Varshitha | Year: 2026

1. PROJECT OVERVIEW

SpendWiser is a full-stack MERN (MongoDB, Express, React, Node.js) web application designed for personal financial management, built specifically keeping Indian users in mind. The application provides a complete solution for tracking daily income and expenses, managing multiple financial wallets (such as a bank account, cash, and savings), automatically parsing bank SMS messages to extract transaction details, receiving AI-powered spending insights via Google Gemini, and viewing personalized investment recommendations based on live government interest rates. The project is organized as a monorepo with a React and TypeScript frontend in the client/ folder and a Node.js and Express backend in the backend/ folder. It is configured for one-click deployment on Vercel using a vercel.json routing file that bridges the frontend and backend traffic seamlessly.

2. TECHNOLOGY STACK

Frontend

The frontend is built using React 19 with TypeScript, ensuring fully type-safe components, props, and state. All data types such as Transaction, Wallet, User, and Budget are defined in a central types.ts file and used consistently across every component. Vite is the build tool, chosen for its extremely fast Hot Module Replacement during development. Tailwind CSS handles all styling with a custom dark cyberpunk theme that uses glassmorphism effects, glowing text, and smooth reveal animations defined as custom keyframes. Recharts is used for the spending bar chart on the dashboard, and a custom LiquidGauge SVG component (built from scratch) displays animated budget usage gauges. Lucide Icons provides all the interface icons throughout the application.

Backend

The backend is a Node.js and Express REST API server. Data is stored in MongoDB, accessed through the Mongoose ODM which provides schema definitions, automatic type validation, and middleware hooks. User passwords are never stored in plain text — they are hashed using bcryptjs with a salt factor of 10 through a Mongoose pre-save hook that fires automatically before every save operation. Authentication is handled using JWT (JSON Web Tokens): after a successful login, the server signs a token using a secret key, and the client includes this token in the Authorization header of every API request. The auth middleware on the backend verifies the token on every protected route without touching the database, making the system stateless and scalable. Rate limiting via express-rate-limit restricts login attempts to 5 per 15 minutes per IP address to prevent brute-force attacks. Request bodies are validated using Joi schemas before reaching any controller. All transactions are also automatically backed up to an Excel file using the ExcelJS library, downloadable from the backend.

AI and External Services

Google Gemini AI is integrated through the official @google/genai SDK on the frontend. It powers two views: the Gemini Core (Insights) view, which generates a narrative spending analysis by sending the transaction list to Gemini with a structured prompt, and the AI Advisor view, which requests a strict JSON response containing saving tips, investment recommendations, budget optimizations, and a financial health score from 0 to 100. A backend web scraper service (scraperService.js) fetches live interest rates for PPF, NPS, Fixed Deposits, and Sukanya Samriddhi Yojana from official government websites. This data is stored in the MongoDB Rates collection and refreshed bi-monthly via a node-

cron job, so every API request serves cached data instantly without live scraping. Optionally, Twilio can be configured to forward incoming bank SMS messages to the /api/sms/webhook endpoint for fully automatic transaction parsing.

3. DATABASE DESIGN

The MongoDB database contains four main collections. The User collection stores each user's id, email (stored in lowercase), bcrypt-hashed password, name, and timestamps. The Transaction collection stores the amount, type (income or expense), category, description, date, walletId linking to the user's wallet, currency (defaulting to INR), and the source of the transaction (manual, sms, or api). Transaction IDs are generated using UUID v4. The BankDetails collection stores bank account information securely in MongoDB — account numbers are masked in the UI and only the last four digits are ever displayed to the user. The Rates collection holds the scraped government interest rates and is updated automatically every two months by the cron job.

4. REST API ENDPOINTS

The backend exposes four groups of API routes. The auth routes under /api/auth handle user registration (POST /register), login (POST /login), fetching a user profile (GET /user/:userId), and updating a user profile (PUT /user/:userId). All auth routes apply Joi validation and rate limiting. The transaction routes under /api/transactions support fetching all transactions for the logged-in user (GET), creating a new transaction (POST), updating an existing one (PUT /:id), deleting a transaction (DELETE /:id), and downloading the Excel backup file (GET /download). The bank detail routes under /api/bank-details handle saving, retrieving, and deleting bank account information. The SMS route at POST /api/sms/webhook receives raw SMS text from Twilio and automatically creates a transaction after parsing. There is also a GET /api/investment-rates/scrape endpoint that returns the cached government interest rates from MongoDB instantly.

5. ALL PAGES — DETAILED EXPLANATION

Login and Register Page

The application opens to a login page when no active session is found in localStorage. A single component handles both login and registration using an isLoginMode toggle. On login, the AuthService calls POST /api/auth/login, which validates the request body via Joi, checks the bcrypt-hashed password, and returns a JWT along with the user object. The client stores the JWT in localStorage and all future API calls include it as a Bearer token. On registration, a new User document is created in MongoDB with the automatically hashed password. Error messages (wrong password, rate limit exceeded) are shown in an alert box and a loading spinner appears on the button during the API call.

Dashboard — Neural Overview

The Dashboard is the main page of the application and loads immediately after login. It displays three KPI metric cards at the top: the Net Balance (total income minus total expenses), Total Inflow (sum of all income transactions), and Total Outflow (sum of all expenses). These values are computed using React's useMemo hook, which recalculates only when the transactions array changes, optimizing performance. Below the metrics sits the Budget Pulse section, which renders each budget category as an animated LiquidGauge — the liquid fill level rises as spending approaches the monthly limit. A Recharts BarChart visualizes expenses grouped by category, with data computed by reducing expense transactions into a category-keyed object. A Live Feed panel shows the most recent transactions in real-time, searchable via a sticky search bar. A CSV Export button generates and downloads a spreadsheet entirely on the client side using the browser Blob API.

Ledger Feed

The Ledger Feed page shows the complete transaction history sorted from newest to oldest. Transactions can be searched by description or category. Each entry displays the date, category, amount in rupees, and the source badge (manual, SMS, or API). A delete button on each row removes the transaction optimistically from React state and then fires a DELETE /api/transactions/:id request to MongoDB. An Export Excel button calls GET /api/transactions/download to fetch the data.xlsx backup file.

Gemini Core and AI Advisor

The Gemini Core (Insights) page sends the user's full transaction list to Google Gemini AI and displays a narrative spending analysis — identifying the biggest categories, flagging unusual patterns, and suggesting savings. The AI Advisor page makes a more structured request, asking Gemini to respond with a strict JSON object containing saving_tips, investment_recommendations, budget_optimizations, and a financial_health_score. The JSON is parsed on the client and rendered as color-coded advice cards. Both views show a loading spinner while awaiting the Gemini API response and handle errors gracefully if the response is malformed.

Investment Plans

The Investment Plans page fetches live government interest rates from the backend via GET /api/investment-rates/scrape and displays investment recommendation cards for PPF at 7.1% per annum, NPS at 8 to 10%, Fixed Deposits at 7.5%, and Sukanya Samriddhi Yojana at 8.2%. The app calculates the user's average monthly savings from their transaction data and personalizes the recommended

investment amounts for each instrument. Since the rates are scraped from official government websites and cached in MongoDB, the response is instant on every page load.

User Profile and Settings

The User Profile page allows users to update their name and email and to save multiple bank account details (bank name, account number, IFSC code) which are stored securely in MongoDB. Account numbers are always masked in the UI — only the last four digits are visible. The Settings page provides wallet management (add, edit, or delete wallets), monthly budget limit configuration per category, and data export and import options for full transaction backups in JSON format.

Transaction Modal, SMS Parser, and SMS Setup

The "Initialize Log" modal is a global overlay that opens from the top header and provides a form to manually enter a transaction with fields for amount, type, category, description, date, and wallet. The "Decrypt SMS" modal lets users paste a raw bank SMS message. The backend parses it using regular expressions to extract the amount (matching patterns like "INR 1,200.00" or "Rs. 500"), the transaction type from the words "debited" or "credited", and the merchant name. The extracted data pre-fills the transaction form for the user to confirm. The "SMS Setup" modal provides a step-by-step Twilio configuration guide so bank messages are automatically forwarded to the webhook and logged without any manual input.

---

6. COMPLETE USER WORKFLOW

A new user opens SpendWiser and sees the login page because no session exists in localStorage. They switch to registration mode, enter their name, email, and password, and submit the form. The backend validates the request, hashes the password using bcrypt, stores the new User document in MongoDB, and returns a JWT. The frontend stores the token and loads the full application. The dashboard shows zero balances. The user clicks "Initialize Log," enters a ¹50,000 salary as income, assigns it to the Main Checking wallet, and saves. React state updates instantly (optimistic update) and the dashboard shows the new balance before the API response even arrives. The POST /api/transactions request saves the transaction to MongoDB and the Excel backup simultaneously. The user then clicks "Decrypt SMS," pastes a Zomato debit message, and the app auto-extracts ¹1,200 as a Food expense. In the AI Advisor tab, clicking "Get AI Suggestions" sends all transactions to Gemini, which returns advice to reduce food delivery spending and start a monthly SIP. Finally, in Investment Plans, the user sees a live recommendation to invest ¹2,000 per month into PPF at the current 7.1% rate.

---

7. KEY TECHNICAL POINTS FOR INTERVIEW

SpendWiser demonstrates several core software engineering concepts. JWT authentication is stateless — the server signs a token with a secret key and the client sends it with every request, so no session data is stored on the server, making the system easily scalable. Password security is implemented using bcryptjs with a Mongoose pre-save hook so hashing is automatic and passwords are never stored in readable form. The Mongoose schema for User includes a comparePassword instance method that calls bcrypt.compare() to verify credentials safely.

On the frontend, optimistic UI updates make the application feel instant — React state is updated immediately when a transaction is added, and only rolls back if the API call fails. The useMemo hook

is used in the Dashboard to avoid recalculating income totals and category groupings on every render. The AI integration uses prompt engineering — the Gemini API prompt is carefully worded to request either a narrative paragraph or a strict JSON schema, and the response is parsed with error handling. The web scraper and node-cron pattern shows backend automation: rates are fetched from government websites once every two months and served from a MongoDB cache on every subsequent request, keeping the API fast while the data remains current. Together, these features make SpendWiser a well-rounded full-stack project that covers authentication, database design, REST API architecture, AI integration, real-time UI updates, and automated background jobs.